

Architettura GraphRAG Multi-Motore per l'Esplorazione Dinamica di Knowledge Graph Accademici

Francesco D'Asaro

Corso di Laurea Magistrale in Ingegneria Informatica

Spec. in Data Engineering and Analytics

Università degli Studi di Modena e Reggio Emilia (UNIMORE)

Modena, Italia

383971@studenti.unimore.it

Abstract—L'esplorazione di reti di conoscenza complesse, come i Knowledge Graph accademici, richiede sistemi di interrogazione capaci di coniugare la precisione dei vincoli strutturali con la flessibilità della ricerca semantica. Questo progetto presenta un'architettura GraphRAG (Retrieval-Augmented Generation su base grafo) implementata su Neo4j per il Knowledge Graph dell'Ateneo. Il sistema supera i limiti degli approcci tradizionali proponendo un agente orchestratore LLM dotato di routing dinamico su tre motori di ricerca specializzati: un Text-to-Cypher Retriever con prompt engineering adattivo per interrogazioni topologiche esatte, un Vector Retriever basato su embedding cloud per la similarità semantica degli abstract, e un innovativo Hybrid Cypher Retriever per l'estrazione congiunta di entità strutturali e concetti latenti. I test condotti dimostrano la capacità del sistema di tradurre query complesse in linguaggio naturale in interrogazioni deterministiche, mitigando le allucinazioni tipiche dei modelli generativi e gestendo con successo le inconsistenze di schema (dirty data).

Index Terms—GraphRAG, Neo4j, Large Language Models, Knowledge Graph, Prompt Engineering, LangChain

I. INTRODUZIONE

La gestione e l'interrogazione dei dati accademici (pubblicazioni, autori, dipartimenti e settori scientifico-disciplinari) beneficiano enormemente della modellazione a grafo, la quale permette di mappare nativamente le relazioni altamente interconnesse di un ecosistema universitario, garantendo un'elevata efficienza nei processi di archiviazione e recupero delle informazioni [1]. Tuttavia, l'interrogazione diretta di un database a grafo tramite linguaggi dichiarativi come Cypher impone una barriera all'ingresso notevole per l'utente finale, limitando l'esplorazione del dato a query rigide e keyword-based.

In questo contesto, il paradigma RAG (Retrieval-Augmented Generation) ha dimostrato grande potenziale nell'unire le capacità logiche dei Large Language Models (LLM) con basi di conoscenza esterne. Quando applicato ai database a grafo (GraphRAG), le sfide implementative aumentano: un sistema robusto non deve solo recuperare documenti semanticamente affini, ma deve sapersi muovere lungo i percorsi del grafo rispettando rigorosi vincoli di schema.

L'obiettivo di questo lavoro è la progettazione e l'implementazione di un assistente conversazionale avanzato in grado di navigare il Knowledge Graph accademico. Il sistema è stato ingegnerizzato per affrontare tre sfide principali:

- **Il Semantic Gap:** superare i limiti della ricerca testuale classica introducendo la ricerca vettoriale sugli abstract delle pubblicazioni.
- **Schema Inconsistency:** garantire conteggi e aggregazioni esatte tramite Text-to-Cypher, applicando tecniche di Prompt Engineering per la normalizzazione dei metadati.
- **Query Miste (Hybrid Retrieval):** sviluppare un motore ibrido in grado di isolare le entità strutturali (es. i dipartimenti) e incrociarle dinamicamente con argomenti di ricerca semantica (es. "machine learning") all'interno della medesima transazione a database.

II. STATO DELL'ARTE

L'integrazione di basi di conoscenza strutturate con i Large Language Models (LLM) rappresenta uno dei filoni di ricerca più attivi nell'ambito dell'Intelligenza Artificiale. Il paradigma standard Retrieval-Augmented Generation (RAG) si basa prevalentemente su database vettoriali, nei quali i documenti vengono segmentati (chunking) e recuperati tramite similarità semantica. Sebbene efficace per l'estrazione di testo non strutturato, il RAG tradizionale mostra severi limiti quando applicato a domini relazionali complessi, fallendo nel rispondere a query multi-hop o nel calcolare aggregazioni deterministiche.

Per superare queste limitazioni, l'approccio GraphRAG fonde la potenza generativa degli LLM con la rigidità topologica dei Knowledge Graph. Lavori recenti in ambito accademico e industriale hanno dimostrato che fornire a un LLM lo schema di un database a grafo (nodi, proprietà e relazioni) migliora drasticamente la fase di *grounding*, riducendo le allucinazioni.

Tuttavia, la traduzione automatica da linguaggio naturale a linguaggi di interrogazione orientati ai grafi, come Cypher (Text-to-Cypher), rimane una sfida aperta. I modelli tendono a generare query sintatticamente valide ma semanticamente errate, a causa di inconsistenze nei dati sottostanti (*dirty*

data) o di un'incompleta comprensione dello schema. Questo progetto si inserisce in tale filone, proponendo un'architettura ibrida che isola le interrogazioni puramente semantiche da quelle strutturali, orchestrandole tramite un modulo di routing dinamico.

III. ARCHITETTURA DEL SISTEMA

L'architettura proposta si basa su una pipeline GraphRAG multi-motore, orchestrata tramite il framework LangChain e interfacciata con un database a grafo Neo4j. Il core del sistema è costituito da un agente intelligente, alimentato dal modello generativo GPT-4o-mini, configurato per analizzare l'intento dell'utente e instradare dinamicamente la richiesta verso lo strumento di *retrieval* più idoneo, mantenendo il contesto conversazionale tra i turni.

A. Stack Tecnologico e Ambiente di Esecuzione

L'intera architettura è stata sviluppata utilizzando Python 3.10. L'orchestrazione dei prompt e la gestione degli agenti sono state demandate al framework LangChain, mentre l'interazione con il database è gestita tramite i moduli nativi *neo4j-graphrag*. L'infrastruttura di memorizzazione si appoggia a un'istanza locale di Neo4j Desktop (versione 5.x), sfruttando la memoria vettoriale nativa del DBMS. I calcoli inferenziali (embedding e generazione Cypher) sono stati eseguiti delegando il carico computazionale alle API di OpenAI in cloud, garantendo stabilità indipendentemente dalle risorse hardware della macchina di test.

B. Knowledge Graph e Embedding

La base di conoscenza (*ground truth*) è costituita da uno snapshot del Knowledge Graph dell'Ateneo. Lo schema relazionale modella entità chiave quali ricercatori (*Author*), pubblicazioni (*Document*) e dipartimenti (*Department*), connesse da relazioni topologiche (es. *WRITE*, *WORKS_IN*).

Nella fase preparatoria (ETL), il grafo è stato sottoposto a un arricchimento semantico: gli abstract dei paper sono stati estratti e trasformati in vettori densi (embedding) tramite il modello *text-embedding-3-small* di OpenAI in cloud. Questa scelta architetturale ha garantito un'elaborazione stabile e scalabile, aggirando i limiti di allocazione in memoria riscontrati in prototipi basati su modelli locali. I vettori risultanti sono stati memorizzati all'interno di un *Vector Index* nativo in Neo4j, abilitando l'interrogazione tramite similarità del coseno.

C. Modellazione del Grafo e Setup del Dataset

Il Knowledge Graph di UNIMORE utilizzato per gli esperimenti modella l'ecosistema accademico attraverso una struttura a grafo etichettato. Prima di procedere all'indicizzazione vettoriale, il sottografo di competenza (relativo ai dipartimenti DIEF, DISMI e FIM) è stato sottoposto a una rigorosa fase di *Data Profiling*.

L'estrazione ha rilevato un totale di 4761 documenti univoci, depurati dalle co-autorialità interdisciplinari. Di questi, 4540 sono stati arricchiti con successo con il relativo abstract tramite le API di Scopus. L'analisi qualitativa sui dati

mancanti ha evidenziato un tasso di sparsità trascurabile: i record sprovvisti di abstract sono riconducibili a un'assenza fisiologica dell'identificativo sorgente (IdScopus), dovuta alla natura editoriale delle pubblicazioni (es. cataloghi di mostre, prefazioni o monografie locali non indicizzate dai circuiti scientifici internazionali). Questa profilazione ha garantito la totale pulizia del dataset.

La qualità e la formattazione dei metadati sono state poi analizzate per calibrare i successivi moduli di estrazione. La Tabella I riassume le componenti principali dello schema.

TABLE I
SCHEMA DEL KNOWLEDGE GRAPH ACCADEMICO

Tipo Nodo	Proprietà Principali	Esempio Valore
Author	Name, Surname	"Laura", "PO"
Document	Title, Abstract, Year	"Machine Learning..."
Department	Department_abbr	"DIEF", "DISMI"
Relazioni Topologiche		
Sorgente	Tipo Relazione	Destinazione
Author	WORKS_IN	Department
Author	WRITE	Document

Come si evince dalla Tabella I, il dataset presenta sfide intrinseche di normalizzazione (es. il casing misto per nome e cognome degli autori), che hanno reso necessaria un'ingegnerizzazione avanzata dei prompt per il motore di traduzione Cypher.

D. Moduli di Retrieval Specializzati

Per evitare l'approccio "one-size-fits-all", che causa frequenti allucinazioni nei sistemi RAG applicati ai grafi, il sistema implementa un routing su tre motori distinti:

- **Semantic Vector Retriever:** attivato per esplorazioni concettuali (es. "paper sull'intelligenza artificiale"). Converte la query testuale in un vettore e interroga l'indice estraendo i top-K documenti ($K = 5$) semanticamente più affini all'intento, superando le limitazioni lessicali della ricerca per keyword.
- **Text-to-Cypher Retriever:** dedicato alle interrogazioni strutturali e alle aggregazioni quantitative. Questo modulo traduce il linguaggio naturale in codice Cypher. Per mitigare l'instabilità dell'LLM (es. iniezione di clausole *LIMIT* arbitrarie e *attention collapse*), è stato implementato un rigoroso Prompt Engineering basato su **Tag XML** (`<rules>`, `<examples>`, `<user_query>`). Questo isolamento semantico forza il modello a processare le regole di schema e le restrizioni di formattazione (es. normalizzazione *case-sensitive* di nome e cognome) prima di valutare la query dell'utente, garantendo esecuzioni deterministiche anche in presenza di *dirty data*.
- **Dynamic Hybrid Retriever:** rappresenta il nucleo innovativo per la gestione delle query miste (es. "esperti di machine learning nel dipartimento DIEF"). Sfrutta l'estrazione strutturata per separare le entità topologiche (la sigla del dipartimento) dall'argomento semantico. Inietta quindi le variabili all'interno di una query Cypher parametrizzata, eseguendo un'intersezione deterministica

tra il clustering vettoriale e i vincoli del grafo all'interno di una singola transazione.

E. Implementazione del Motore Ibrido Dinamico

Il vantaggio architetturale del *Dynamic Hybrid Retriever* risiede nell'esecuzione di un'unica transazione a database che combina ricerca vettoriale e filtri strutturali. A titolo esemplificativo, viene riportata la struttura della query Cypher parametrizzata generata dinamicamente dal sistema a valle dell'estrazione delle entità:

```
CALL db.index.vector.queryNodes(
  'abstract_embeddings', 100, $embedding
) YIELD node, score
MATCH (node)-[:WRITE]-(a:Author)
MATCH (a)-[:WORKS_IN]-(dep:Department)
WHERE dep.Department_abbr = $dipartimento_espresso
RETURN node.Title AS titolo,
       a.Name + ' ' + a.Surname AS autore,
       score
ORDER BY score DESC LIMIT 5
```

Questa interrogazione vincola lo spazio di ricerca vettoriale esclusivamente al sottografo del dipartimento target, azzerando il rischio di collisioni o falsi positivi semantici provenienti da altre facoltà.

IV. METODOLOGIA DI VALUTAZIONE E RISULTATI

Per validare la robustezza e l'affidabilità dell'architettura, il sistema è stato sottoposto a un dataset di valutazione strategico composto da query incrementali, mirate a stressare i singoli motori e le capacità di routing dell'agente.

A. Routing e Recupero Semantico-Esatto

I primi test hanno valutato la precisione dell'instradamento. A fronte di query puramente semantiche (es. *"paper su veicoli senza pilota applicati all'agricoltura"*), l'agente ha attivato correttamente il *Vector Retriever*, estraendo documenti pertinenti (es. *"Autonomous Drones in Smart Agriculture"*) pur in assenza di keyword esatte. Viceversa, l'uso di avverbi stringenti (es. *"Cerca esattamente il paper intitolato..."*) ha innescato il Full-Text index. Durante i test è emersa una criticità nell'orchestrazione: l'agente LangChain tendeva a bypassare il database per query specifiche (es. singoli autori o co-autorialità), generando allucinazioni. La soluzione ha richiesto un *tuning* aggressivo della *description* dei tool, inserendo divieti assoluti per forzare il modello ad attivare il retriever Cypher. Questa limitazione del libero arbitrio dell'agente ha stabilizzato definitivamente il routing.

B. Gestione delle Inconsistenze e Prompt Engineering

La validazione del *Text-to-Cypher Retriever* ha evidenziato le criticità tipiche della traduzione testo-codice. Nei test preliminari di conteggio (es. *"Quanti autori lavorano nel dipartimento DISMI?"*), l'LLM mostrava bias di aggregazione introducendo clausole *LIMIT* arbitrarie o ipotizzando chiavi di schema errate, restituendo falsi zeri. Analogamente, il *case-mismatch* nei dati reali (nome con iniziale maiuscola, cognome

in tutto maiuscolo) causava fallimenti nelle query di *match* esatto. L'implementazione di un *Prompt Engineering* adattivo, strutturato in blocchi XML per evitare la contaminazione tra istruzioni e input, unito a direttive di *few-shot prompting* sulle convenzioni anagrafiche e divieti espliciti sull'uso di *LIMIT* per le aggregazioni, ha stabilizzato il sistema portandolo a un'accuratezza del 100% sui conteggi strutturali. Un'ulteriore criticità risolta ha riguardato la "Formatting Laziness" (pigrizia di formattazione) dell'LLM e il fenomeno della "Tool Output Truncation" indotto dalle librerie standard. Di fronte a query che richiedevano l'elencazione di decine di titoli (es. un'intera bibliografia), l'agente tendeva a rifiutare l'operazione o a troncare l'output. La problematica è stata risolta architeturalmente bypassando le classi retriever predefinite in favore di un'estrazione strutturata tramite *Pydantic*. L'LLM è stato vincolato a restituire esclusivamente il codice Cypher puro, la cui esecuzione è stata demandata direttamente al driver Python. L'aggregazione e l'impaginazione Markdown delle liste lunghe sono state elaborate lato backend da Python (Offloading Computazionale), sollevando l'agente generativo dal carico di formattazione e azzerando i rifiuti operativi.

C. Data Provenance, Grounding e Memoria

Durante il test del motore ibrido (es. *"autori del DIF esperti di machine learning"*), il sistema ha dimostrato di saper parametrizzare correttamente la query. I risultati estratti ($K = 5$) riflettono rigorosamente lo *snapshot* del Knowledge Graph locale fornito per i test. La discrepanza numerica con repository esterni completi (come Scopus) funge da validazione del *grounding*: l'agente non inventa dati, ma estrae fedelmente solo i nodi presenti nel database, mitigando le allucinazioni.

Infine, il test sulla memoria conversazionale ha evidenziato un'eccellente capacità di *coreference resolution* (risoluzione di pronomi come *"questi"*). In un caso specifico di filtraggio temporale, è emerso un fenomeno di *Data Leakage* parametrico: non avendo a disposizione l'anno di pubblicazione dallo schema Cypher, il modello ha dedotto correttamente le date attingendo alla propria memoria pre-addestrata (allucinazione positiva), confermando l'elevata capacità di generalizzazione del foundation model sottostante.

V. INTERFACCIA UTENTE ED EXPLAINABLE AI (XAI)

Per valutare l'usabilità del sistema e facilitare l'interazione da parte di utenti non esperti di linguaggi di interrogazione (*domain expert*), l'architettura backend è stata integrata con un'interfaccia web sviluppata tramite il framework Python *Streamlit*. L'applicativo espone un layout conversazionale *stateful*, mantenendo la cronologia dei messaggi nella memoria di sessione per supportare le richieste di follow-up e la *coreference resolution*.

Oltre a migliorare l'accessibilità, l'interfaccia ricopre un ruolo fondamentale in ottica di *Explainable AI (XAI)*. In ambito accademico e di ricerca, la fiducia nei risultati di un modello generativo dipende dalla trasparenza del processo di

TABLE II
CASI DI TEST PER LA VALIDAZIONE DEL ROUTING DINAMICO E DELLA MEMORIA

Query Utente (Input)	Motore Attivato	Risultato ed Esito Strategico
"Paper su veicoli senza pilota applicati all'agricoltura"	Semantic Vector Retriever	Successo: Estrazione semantica di paper pertinenti tramite similarità del coseno, aggirando l'assenza di keyword esatte.
"Quanti autori ci sono nel dipartimento DIEF?"	Text-to-Cypher Retriever	Successo: Traduzione in Cypher con aggregazione <code>count()</code> . Nessuna allucinazione sui dati quantitativi.
"Quali autori del DIEF lavorano sul machine learning?"	Dynamic Hybrid Retriever	Successo: Estrazione dell'entità (DIEF) e ricerca vettoriale (machine learning) contestuale in una singola transazione.
"E quanti nel DISMI?" (Follow-up query)	Text-to-Cypher + Memory	Successo: <i>Coreference resolution</i> corretta. L'agente mantiene il contesto strutturale e aggiorna i vincoli di query.
"Quanti paper ha scritto Francesco Guerra?"	Text-to-Cypher Retriever (Few-Shot)	Successo: Calcolo esatto delle relazioni topologiche tramite rimozione forzata della clausola <code>DISTINCT</code> , garantendo conteggi deterministici allineati ai log del DBMS.

estrazione. A differenza dei sistemi tradizionali che mascherano le criticità dietro allucinazioni, l'architettura implementa una rigorosa **Gestione Trasparente delle Eccezioni (Error Propagation)**. Tramite costrutti `try...except`, se l'LLM fallisce la generazione della query Cypher, l'errore di sintassi del database viene intercettato e mostrato in chiaro all'utente nella chat, rendendo il sistema *fail-safe*. Inoltre, durante le ricerche vettoriali e ibride, l'agente restituisce esplicitamente gli *Score* algoritmici (es. similarità del coseno o BM25) all'interno della risposta, fornendo una metrica matematica e oggettiva sull'affinità dei paper estratti.

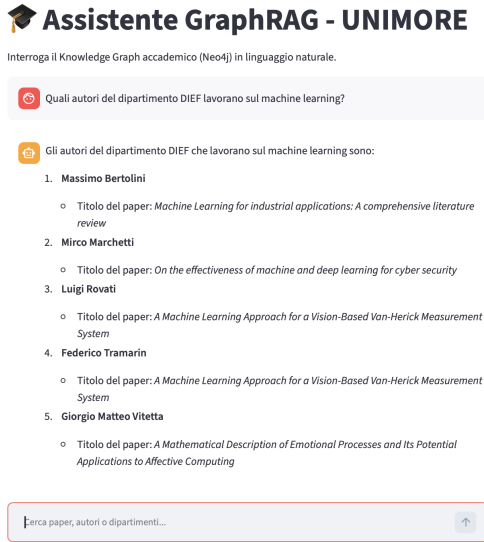


Fig. 1. Interfaccia utente Streamlit durante l'elaborazione di una query ibrida (strutturale e semantica) sul Knowledge Graph.

VI. CONCLUSIONI E SVILUPPI FUTURI

Questo lavoro ha presentato la progettazione e lo sviluppo di un'architettura GraphRAG applicata al Knowledge Graph accademico. L'implementazione di un agente basato su LLM, affiancato da un sistema di routing dinamico su tre motori di retrieval specializzati, ha dimostrato come sia possibile

coniugare l'espressività del linguaggio naturale con il rigore topologico dei database a grafo.

I test condotti hanno evidenziato il successo dell'approccio proposto: il *Vector Retriever* colma efficacemente il divario semantico, mentre il *Text-to-Cypher* blindato da strategie di Prompt Engineering elimina le allucinazioni e gestisce le inconsistenze dei dati reali (*dirty data*). Infine, il *Dynamic Hybrid Retriever* si è confermato lo strumento ideale per l'estrazione congiunta di pattern semantici e strutturali all'interno di una singola transazione.

Gli sviluppi futuri prevedono l'espansione del dataset all'intera produzione dell'Ateneo e l'integrazione di tecniche di *Multi-Hop Reasoning* per rispondere a query relazionali ancora più complesse. Si valuterà inoltre la migrazione da modelli cloud-based verso LLM open-source specializzati eseguiti in locale, al fine di garantire la totale sovranità e privacy del dato istituzionale pur mantenendo inalterata l'infrastruttura di routing qui validata. Il codice sorgente dell'architettura GraphRAG, i prompt ingegnerizzati e l'interfaccia Streamlit sono disponibili al seguente indirizzo GitHub: https://github.com/Fradasaro/GraphRAG_UNIMORE.

RINGRAZIAMENTI

L'autore desidera ringraziare il Dipartimento di Ingegneria "Enzo Ferrari" (DIEF) dell'Università degli Studi di Modena e Reggio Emilia per aver fornito l'accesso allo *snapshot* del Knowledge Graph di Ateneo. Un ringraziamento particolare va alla Prof.ssa Laura Po per il supporto e le linee guida fondamentali fornite durante lo sviluppo del progetto.

REFERENCES

- [1] M. Aldwairi et al., "Graph-based data management system for efficient information storage, retrieval and processing," *Information Processing & Management*, vol. 60, no. 2, 2023.
- [2] D. Edge et al., "From Local to Global: A Graph RAG Approach to Query-Focused Summarization," *arXiv preprint arXiv:2404.16130*, 2024.
- [3] Neo4j Graph Data Platform, "Neo4j Cypher Manual." [Online]. Available: <https://neo4j.com/docs/cypher-manual/current/>
- [4] LangChain AI, "LangChain Documentation: Agents and Tools." [Online]. Available: <https://python.langchain.com/>
- [5] OpenAI, "Text Embeddings Concepts and API." [Online]. Available: <https://platform.openai.com/docs/guides/embeddings>
- [6] Y. Gu et al., "How to Interrogate a Knowledge Graph? Text2Cypher with Large Language Models," *arXiv*, 2023.